

Day 8: ARM Cortex-M Architecture

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

ARM Cortex-M4 Core in nRF52840

The nRF52840 uses an **ARM Cortex-M4F** core (the 'F' means it has a hardware **Floating Point Unit**):

- **64 MHz** maximum clock speed
- **32-bit** registers (R0–R15), 16 general-purpose registers
- **Thumb-2** instruction set (16 and 32-bit instructions mixed for code density)
- **Hardware FPU** for `float` operations without software emulation
- **Harvard architecture** internally but **unified address space** externally

nRF52840 Memory Map

```
0x00000000 - 0x000FFFFF : Flash      (1 MB)  - code, constants
0x10001000 - 0x10001FFF : UICR       (4 KB)  - user config registers
0x20000000 - 0x2003FFFF : RAM        (256 KB) - stack, heap, globals
0x40000000 - 0x5FFFFFFF : Peripherals - UART, SPI, I2C, GPIO...
0xE0000000 - 0xFFFFFFFF : Cortex-M system - NVIC, SysTick, SCB, DWT
```

NVIC — Nested Vectored Interrupt Controller

The NVIC manages all hardware interrupts. Key concepts:

- **Priority levels:** 0–7 on Cortex-M4 (0 = highest priority)
- **Preemption:** a higher-priority ISR can interrupt a lower-priority ISR
- **Pending:** the NVIC can queue an interrupt while a same/lower priority ISR runs

```
NVIC_SetPriority(UART0_IRQn, 3);    // Set UART0 IRQ to priority 3
NVIC_EnableIRQ(UART0_IRQn);        // Enable the IRQ
```

Zephyr configures NVIC priorities automatically based on `CONFIG_*_IRQ_PRIORITY` Kconfig options.

SysTick Timer

SysTick is a 24-bit countdown timer built into every Cortex-M core. Zephyr uses it as its **tick source** — it fires `CONFIG_SYS_CLOCK_TICKS_PER_SEC` times per second (default 100Hz on nRF52840) to drive `k_msleep()`, `k_uptime_get()`, and thread scheduling.

Vector Table

The **vector table** is an array of 32-bit function pointers stored at Flash address 0x00000000:

```
[0] : Initial stack pointer value
[1] : Reset handler address      (entry point after reset/power-on)
[2] : NMI handler
[3] : HardFault handler
...
[N] : Peripheral IRQ handlers
```

Zephyr defines the complete vector table in `arch/arm/core/cortex_m/vector_table.S`.

Fault Handlers

When the CPU encounters an error (invalid memory access, divide-by-zero, bad instruction), it triggers a **fault exception**:

- **HardFault**: catch-all for unhandled faults
- **MemManage**: MPU access violation
- **BusFault**: bad memory bus access (e.g., unaligned access with strict alignment)
- **UsageFault**: undefined instruction, divide-by-zero (if enabled)

Zephyr's fault handler (`z_arm_fault`) prints a register dump and backtrace. Enable with `CONFIG_FAULT_DUMP=2`.

DWT Cycle Counter

The **Data Watchpoint and Trace (DWT)** unit has a 32-bit cycle counter (`DWT->CYCCNT`) that counts CPU clock cycles. Use it to measure execution time with nanosecond resolution:

```
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk; // Enable counter
uint32_t t0 = DWT->CYCCNT;
my_function();
uint32_t cycles = DWT->CYCCNT - t0; // cycles / 64MHz = time in microseconds
```

Practice Tasks

1. Read the nRF52840 Product Specification (PS) section on memory map — find the GPIO register addresses
2. Use the DWT cycle counter to measure how long `memset(buf, 0, 1024)` takes in CPU cycles
3. Deliberately cause a HardFault (dereference address 0x00000000) — read the Zephyr fault dump
4. Use `NVIC_GetPriority()` to print the priority of 5 different IRQs on nRF52840

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day08
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day08/main.cpp](#)